



TOPORAY
Levantamientos topográficos

DOCUMENTACIÓN DEL SISTEMA

MANUAL DEL PROGRAMADOR

Arquitectura, cálculo, persistencia, mantenimiento y despliegue

Campo	Información
Versión	1.0
Fecha de emisión	28 de julio de 2026
Producto	Software web para levantamientos topográficos
Estado	Documento técnico de referencia

Documento preparado para consulta académica, operación controlada y mantenimiento del sistema.

Control documental

Este manual documenta la implementación de TOPORAY versión 1.0. Está dirigido a desarrolladores responsables de revisar, mantener, probar o desplegar la aplicación.

Versión	Fecha	Descripción	Estado
1.0	28/07/2026	Emisión inicial del manual	Vigente

Contenido

1. Descripción técnica	3
2. Estructura del proyecto	4
3. Estructura HTML y navegación	5
4. Sistema visual y adaptación responsive	6
5. Modelo de estado	7
6. Validación y ciclo de vida de puntos	8
7. Conversión angular, rumbo y proyecciones	9
8. Geometría de zonas	11
9. Renderizado del plano	12
10. Zonas, referencias y herramientas rápidas	13
11. Persistencia, proyectos e historial	14
12. Importación y exportación	15
13. Informe técnico y escala de dibujo	16
14. Eventos, accesibilidad y comportamiento móvil	17
15. Pruebas y control de calidad	18
16. Mantenimiento, seguridad y limitaciones	19

1. Descripción técnica

TOPORAY es una aplicación web estática de una sola página. No requiere compilador, gestor de paquetes, servidor de aplicaciones ni base de datos. La lógica se ejecuta íntegramente en el navegador.

Capa	Tecnología	Responsabilidad
Estructura	HTML5	Navegación, vistas, formularios, tablas, diálogos y canvases.
Presentación	CSS3	Temas, distribución, tablas fijas y reglas responsive.
Lógica	JavaScript ES6+	Estado, validación, cálculo, dibujo, persistencia y exportación.
Gráfica	Canvas 2D	Plano cartesiano interactivo y plano técnico exportable.
Persistencia	localStorage	Proyecto activo, proyectos nombrados y selección actual.
Intercambio	CSV/TXT/PNG/HTML	Importación, exportación e informe imprimible.

Principio de operación

El objeto global state es la fuente de verdad. `update()` calcula el levantamiento, sincroniza salidas, redibuja el canvas y conserva el estado local.

Compatibilidad objetivo

- Navegadores modernos con Canvas 2D, dialog, structuredClone, Blob y localStorage.
- Diseño adaptable desde 320 px de ancho.
- Ejecución directa desde un servidor HTTP estático o GitHub Pages.
- No existe sincronización multiusuario ni autenticación.

2. Estructura del proyecto

Ruta	Contenido
index.html	Punto de entrada y estructura completa de la interfaz.
styles.css	Variables, componentes, layout, temas, responsive e impresión.
app.js	Estado, motor topográfico, canvas, proyectos y productos de salida.
assets/toporay-*.svg	Identidad visual y favicon.
assets/report-template.png	Plantilla institucional utilizada por el informe.
README.md	Instrucciones básicas de ejecución y resumen funcional.
output/pdf	Manuales finales generados.
tmp/pdfs	Activos y scripts temporales de documentación y revisión.

Ejecución local

Comando

```
python -m http.server 5175
```

Después, abrir <http://127.0.0.1:5175/>.

Despliegue estático

- Publique index.html, styles.css, app.js y assets sin cambiar sus rutas relativas.
- Configure index.html como documento de entrada.
- Use HTTPS en producción para un origen estable de localStorage.
- Evite cambiar el origen o la ruta publicada sin planificar la migración de proyectos del navegador.

3. Estructura HTML y navegación

Las vistas se mantienen en el mismo documento y se identifican mediante `data-view` y `data-view-panel`. `switchView()` activa el panel solicitado y actualiza el estado visual del menú.

Código 1. Navegación principal - index.html

```
21 <nav class="sidebar-nav">
22   <button class="nav-button is-active" type="button" data-view="plan" title="Plano">
23     <span class="nav-icon" aria-hidden="true">☐</span><span class="sidebar-label">Plano</span>
24   </button>
25   <button class="nav-button" type="button" data-view="points" title="Puntos">
26     <span class="nav-icon" aria-hidden="true">•</span><span class="sidebar-label">Puntos</span>
27   </button>
28   <button class="nav-button" type="button" data-view="zones" title="Zonas">
29     <span class="nav-icon" aria-hidden="true">☐</span><span class="sidebar-label">Zonas</span>
30   </button>
31   <button class="nav-button" type="button" data-view="calculations" title="Cálculos">
32     <span class="nav-icon" aria-hidden="true">f</span><span class="sidebar-label">Cálculos</span>
33   </button>
34   <button class="nav-button" type="button" data-view="projects" title="Proyectos">
35     <span class="nav-icon" aria-hidden="true">☐</span><span class="sidebar-label">Proyectos</span>
36   </button>
37   <button class="nav-button" type="button" data-view="files" title="Importar y exportar">
38     <span class="nav-icon" aria-hidden="true">☐</span><span class="sidebar-label">Importar y exportar</span>
39   </button>
40   <button class="nav-button" type="button" data-view="settings" title="Configuración">
41     <span class="nav-icon" aria-hidden="true">☐</span><span class="sidebar-label">Configuración</span>
42   </button>
43 </nav>
```

Código 1. Navegación principal declarada en index.html, líneas 21–43.

- La barra lateral contiene Plano, Puntos, Zonas, Cálculos, Proyectos, Archivos y Configuración.
- Los diálogos nativos gestionan proyectos, puntos y zonas.
- El canvas `#plotCanvas` es el único escenario gráfico interactivo.
- Los atributos `aria-label` y `aria-live` describen controles y mensajes de estado.
- `app.js` se carga al final del `body` para que todos los elementos estén disponibles.

4. Sistema visual y adaptación responsive

styles.css define los temas mediante variables y conserva una interfaz operacional de alta densidad. El tema claro reemplaza los mismos tokens sin duplicar los componentes.

Código 3. Adaptación móvil - styles.css

```
1437 @media (max-width: 650px) {
1438   .topbar {
1439     flex-wrap: wrap;
1440   }
1441
1442   .topbar-brand-symbol {
1443     display: block;
1444   }
1445
1446   .project-title-control {
1447     width: calc(100% - 86px);
1448   }
1449
1450   .active-zone-topbar {
1451     order: 3;
1452     width: calc(100% - 190px);
1453   }
1454
1455   .topbar-actions {
1456     order: 4;
1457     margin-left: auto;
1458   }
1459
1460   .view-host {
1461     padding: 12px 10px 78px;
1462   }
1463
1464   .view-heading {
1465     align-items: flex-start;
1466     flex-direction: column;
1467   }
1468
1469   .view-heading > button {
1470     width: 100%;
```

Código 2. Reglas de adaptación para pantallas pequeñas.

Criterio	Implementación
Layout	Barra lateral fija, cabecera, panel principal y navegación móvil.
Plano	Canvas cuadrado con dimensiones estables y relación de aspecto controlada.
Tablas	Desplazamiento horizontal; zona, punto y acciones permanecen visibles.
Puntos de quiebre	980 px para menú móvil y 650 px para reorganización compacta.
Impresión	Fondo blanco, texto oscuro y ocultamiento de comandos de interacción.

5. Modelo de estado

Código 4. Estado inicial - app.js

```
193 function createBlankState(projectName = "Levantamiento sin nombre") {
194   const mainZone = createZone({ id: makeId("zone"), name: "Zona principal", color: ZONE_COLORS[0] });
195   return {
196     version: 2,
197     projectName,
198     modifiedAt: new Date().toISOString(),
199     station: { east: 1000, north: 1000 },
200     mode: "radiacion",
201     showPoints: true,
202     showLines: true,
203     showGrid: true,
204     showZoneNames: true,
205     showPointNumbers: true,
206     units: "m",
207     gridSize: 0,
208     appearance: "dark",
209     axisDecimals: 0,
210     view: { zoom: 1, panEast: 0, panNorth: 0 },
211     drawingScale: { paper: "A3", orientation: "horizontal", mode: "auto", denominator: 500 },
212     zones: [mainZone],
213     activeZoneId: mainZone.id,
214     observations: [defaultObservation(1, mainZone.id)],
215   };
};
```

Código 4. Estado inicial normalizado de un levantamiento.

Propiedad	Tipo	Responsabilidad
projectName	string	Nombre visible y clave propuesta al guardar.
station	{east,north}	Coordenadas del BM fijo.
mode	string	Base de cálculo: radiación o poligonal.
zones	array	Objetos geométricos, estilo, referencia y visibilidad.
observations	array	GMS, distancia, descripción, UID y zona de cada punto.
view	object	Zoom y desplazamiento del canvas.
drawingScale	object	Papel, orientación, modo y denominador.
show*	boolean	Preferencias de puntos, líneas, cuadrícula y rótulos.

migrateState() completa propiedades faltantes y normaliza proyectos guardados por versiones anteriores. Cada zona y punto usa un UID para conservar identidad durante reordenamientos.

6. Validación y ciclo de vida de puntos

La validación ocurre en el formulario modal, en la edición de tabla y durante `computeSurvey()`. Los controles visuales impiden valores fuera de rango y el motor vuelve a comprobarlos antes de asignar coordenadas.

Campo	Regla
Número	Entero positivo y único dentro de la zona.
Grados	$0 \leq G \leq 360$; con $G = 360$, $M = 0$ y $S = 0$.
Minutos	$0 \leq M < 60$.
Segundos	$0 \leq S < 60$.
Distancia	$d \geq 0$; para producir coordenadas debe ser mayor que 0.
Descripción	Texto libre; se protege en CSV/TXT cuando contiene comas o comillas.

Operaciones sobre una observación

- `normalizeObservation()` convierte tipos y limita ángulos.
- `moveObservation()` cambia el orden dentro de la zona y renumera.
- `clearPointValues()` conserva UID, zona y número, pero reemplaza los demás valores.
- `deleteObservation()` elimina el elemento del array, renumera y mantiene cerrado el polígono.
- `buildInputErrors()` detecta repetidos, secuencia, duplicados de coordenada y geometría cruzada.

Invariante

Una fila solo participa en la geometría cuando tiene ángulo válido y distancia mayor que 0. Una fila vacía puede permanecer para captura posterior sin crear conexiones incorrectas.

7. Conversión angular, rumbo y proyecciones

El motor normaliza los ángulos a una vuelta completa y expresa el azimut en notación cuadrantal para facilitar la revisión topográfica.

Código 5. Conversión GMS y rumbo - app.js

```
330 function dmsToDecimal({ degrees, minutes, seconds }) {
331   return toNumber(degrees) + toNumber(minutes) / 60 + toNumber(seconds) / 3600;
332 }
333
334 function decimalToDms(value) {
335   const normalized = ((value % 360) + 360) % 360;
336   let degrees = Math.floor(normalized);
337   const minuteFloat = (normalized - degrees) * 60;
338   let minutes = Math.floor(minuteFloat);
339   let seconds = Math.round((minuteFloat - minutes) * 60000) / 1000;
340   if (seconds >= 60) {
341     seconds = 0;
342     minutes += 1;
343   }
344   if (minutes >= 60) {
345     minutes = 0;
346     degrees = (degrees + 1) % 360;
347   }
348   return { degrees, minutes, seconds };
349 }
350
351 function formatDms(value) {
352   const dms = decimalToDms(value);
353   return `${dms.degrees}\u00b0 ${String(dms.minutes).padStart(2, "0")}' ${dms.seconds.toFixed(3).padStart(6, "0")}"";
354 }
355
356 function bearingFromAzimuth(azimuth) {
357   const angle = ((azimuth % 360) + 360) % 360;
358   if (angle <= 90) return `N ${formatDms(angle)} E`;
359   if (angle <= 180) return `S ${formatDms(180 - angle)} E`;
360   if (angle <= 270) return `S ${formatDms(angle - 180)} O`;
361   return `N ${formatDms(360 - angle)} O`;
```

Código 5. Conversión GMS, normalización y rumbo.

Operación	Expresión
Azimut decimal	$Az = G + M/60 + S/3600$
Conversión a radianes	$Azrad = Az \times \pi / 180$

Rumbo

bearingFromAzimuth() reduce el ángulo al cuadrante correspondiente y devuelve N/S, el ángulo reducido y E/O.

7.1 Proyecciones y coordenadas cartesianas

Código 6. Proyecciones y coordenadas - app.js

```
460     const angleValid =
461         toNumber(observation.degrees) >= 0 &&
462         toNumber(observation.degrees) <= 360 &&
463         toNumber(observation.minutes) >= 0 &&
464         toNumber(observation.minutes) < 60 &&
465         toNumber(observation.seconds) >= 0 &&
466         toNumber(observation.seconds) < 60 &&
467         !(toNumber(observation.degrees) === 360 && (toNumber(observation.minutes) !== 0 || toNumber(observation.seconds) !== 0));
468     const hasCoordinates = toNumber(observation.distance) > 0 && angleValid;
469     const azimuth = dmsToDecimal(normalized);
470     const radians = (azimuth * Math.PI) / 180;
471     const distance = Math.max(0, toNumber(normalized.distance));
472     const deltaEast = distance * Math.sin(radians);
473     const deltaNorth = distance * Math.cos(radians);
474     const base = state.mode === "poligonal" ? runtime.cursor : runtime.base;
475     const point = {
476         uid: observation.uid,
477         id: String(observation.id || ""),
478         zoneId: zone.id,
479         rowIndex,
480         azimuth,
481         radians,
482         distance,
483         deltaEast,
484         deltaNorth,
485         east: base.east + deltaEast,
486         north: base.north + deltaNorth,
487         bearing: bearingFromAzimuth(azimuth),
488         description: String(observation.description || ""),
489         source: normalized,
490         hasCoordinates,
491         hasData: hasMeasurementData(observation),
492         breakBefore: hasCoordinates && runtime.breakNext,
493         referenceValid: runtime.base.referenceValid,
494         status: "Incompleto",
495     };
```

Código 6. Validación, proyecciones y coordenadas cartesianas.

Operación	Expresión
Proyección Este	$\Delta E = d \times \text{sen}(\text{Azrad})$
Proyección Norte	$\Delta N = d \times \text{cos}(\text{Azrad})$
Coordenada Este	$E = E_{\text{base}} + \Delta E$
Coordenada Norte	$N = N_{\text{base}} + \Delta N$

En radiación, la base corresponde a la referencia de la zona. En modo poligonal, runtime.cursor avanza a la coordenada calculada del punto anterior.

Orientación matemática

El azimut parte del Norte y crece en sentido horario; por ello la proyección Este usa seno y la proyección Norte usa coseno.

8. Geometría de zonas

Código 7. Área y perímetro - app.js

```
621 function polygonArea(points) {
622     if (points.length < 3) return 0;
623     let sum = 0;
624     for (let index = 0; index < points.length; index += 1) {
625         const a = points[index];
626         const b = points[(index + 1) % points.length];
627         sum += a.east * b.north - b.east * a.north;
628     }
629     return Math.abs(sum) / 2;
630 }
631
632 function perimeter(points, close) {
633     if (points.length < 2) return 0;
634     let total = 0;
635     for (let index = 1; index < points.length; index += 1) {
636         total += Math.hypot(points[index].east - points[index - 1].east, points[index].north - points[index - 1].north);
637     }
638     if (close && points.length > 2) {
639         total += Math.hypot(points[0].east - points.at(-1).east, points[0].north - points.at(-1).north);
640     }
641     return total;
}
```

Código 7. Cálculo de área por determinantes y longitud por segmentos.

Tipo de zona	Condición de completitud	Métrica
Polígono	Tres o más puntos válidos, cerrado, sin cruces, duplicados ni vacíos internos.	Área y perímetro.
Línea	Dos o más puntos válidos y continuidad de segmentos.	Longitud.
Puntos	Al menos un punto válido.	Conteo y coordenadas.

- `polygonArea()` aplica la fórmula del cordón o shoelace y devuelve el valor absoluto entre 2.
- `perimeter()` suma distancias euclidianas entre pares consecutivos y agrega el cierre cuando corresponde.
- `polygonHasCrossings()` compara segmentos no adyacentes mediante orientación.
- `buildSegments()` separa tramos cuando existe una fila incompleta entre puntos válidos.
- Las métricas solo se publican cuando el estado de la zona es Completa.

9. Renderizado del plano

Código 8. Plano técnico exportable - app.js

```
1834 function createTechnicalPlanCanvas(survey) {
1835   const canvas = document.createElement("canvas");
1836   canvas.width = 1400;
1837   canvas.height = 1000;
1838   const ctx = canvas.getContext("2d");
1839   const width = canvas.width;
1840   const height = canvas.height;
1841   const station = { east: toNumber(state.station.east), north: toNumber(state.station.north), id: "BM" };
1842   const visibleZoneIds = new Set(state.zones.filter((zone) => zone.visible).map((zone) => zone.id));
1843   const visiblePoints = survey.points.filter((point) => point.hasCoordinates && visibleZoneIds.has(point.zoneId));
1844   const eastValues = [station, ...visiblePoints].map((point) => point.east);
1845   const northValues = [station, ...visiblePoints].map((point) => point.north);
1846   let eastAxis = niceAxis(Math.min(...eastValues), Math.max(...eastValues), station.east);
1847   let northAxis = niceAxis(Math.min(...northValues), Math.max(...northValues), station.north);
1848   const margin = { left: 90, right: 45, top: 45, bottom: 70 };
1849   const plotW = width - margin.left - margin.right;
1850   const plotH = height - margin.top - margin.bottom;
1851   ({ eastAxis, northAxis } = balancedAxes(eastAxis, northAxis, plotW, plotH, station));
1852   const x = (east) => margin.left + ((east - eastAxis.min) / (eastAxis.max - eastAxis.min)) * plotW;
1853   const y = (north) => margin.top + (1 - (north - northAxis.min) / (northAxis.max - northAxis.min)) * plotH;
1854   const colors = plotColors(true);
1855   ctx.fillStyle = colors.background;
1856   ctx.fillRect(0, 0, width, height);
1857   drawGrid(ctx, eastAxis, northAxis, x, y, margin, plotW, plotH, width, height, colors);
```

Código 8. Construcción del canvas técnico con salida blanca.

drawPlot() adapta la resolución del canvas al devicePixelRatio, calcula ejes balanceados y conserva la misma escala gráfica en Este y Norte. plotTransform almacena las funciones de conversión entre coordenadas y píxeles.

Componente	Responsabilidad
niceAxis / axisWithinRange	Generar límites y marcas de cuadrícula legibles.
balancedAxes	Igualar unidades por píxel y evitar deformación del plano.
applyView	Aplicar zoom y desplazamiento sin mover el BM lógico.
drawZoneGeometry	Dibujar polígonos y líneas por zona.
drawPoint / drawLegend	Puntos, BM, numeración y leyenda.
createTechnicalPlanCanvas	Crear PNG e imagen de informe a 1400 × 1000 px, siempre blanca.

10. Zonas, referencias y herramientas rápidas

createZone() normaliza nombre, color, tipo, visibilidad, cierre y referencia. Las referencias admitidas son station, point y custom.

Referencia	Comportamiento
station	Usa E0/N0; corresponde al BM del levantamiento.
point	Usa una observación calculada de otra zona como origen.
custom	Usa coordenadas Este/Norte escritas en el formulario de zona.

Herramientas de canvas

- pointNearCanvasEvent() busca el punto visible más cercano en un radio de 20 px.
- setQuickMode() activa selección, movimiento, limpieza o medición.
- Mover abre el mismo formulario de edición utilizado por la tabla.
- Medir calcula la distancia euclidiana entre dos coordenadas existentes.
- El arrastre del canvas modifica panEast y panNorth; la rueda ajusta zoom entre 0.5 y 20.
- Los interruptores de puntos, líneas y cuadrícula alteran visualización, no el modelo geométrico.

Separación de responsabilidades

La posición de un punto no se modifica directamente por arrastre. Mover significa editar su observación, con lo cual se conserva la trazabilidad del cálculo.

11. Persistencia, proyectos e historial

Código 9. Persistencia local - app.js

```
1930 function loadProjects() {
1931   try {
1932     const parsed = JSON.parse(localStorage.getItem(PROJECTS_KEY));
1933     return parsed && typeof parsed === "object" ? parsed : {};
1934   } catch {
1935     return {};
1936   }
1937 }
1938
1939 function writeProjects(projects) {
1940   localStorage.setItem(PROJECTS_KEY, JSON.stringify(projects));
1941 }
1942
1943 function saveLocalState() {
1944   localStorage.setItem(STORAGE_KEY, JSON.stringify(state));
1945   const normalizedName = normalizeProjectName(state.projectName);
1946   if (loadedProjectName && normalizedName === loadedProjectName) {
1947     const projects = loadProjects();
1948     state.modifiedAt = new Date().toISOString();
1949     projects[loadedProjectName] = clone({ ...state, projectName: loadedProjectName, modifiedAt: state.modifiedAt });
1950     writeProjects(projects);
1951     localStorage.setItem(CURRENT_PROJECT_KEY, loadedProjectName);
1952     els.saveState.textContent = "Guardado automáticamente";
1953   } else if (loadedProjectName) {
1954     els.saveState.textContent = "Nombre nuevo: pulse Guardar";
1955   } else {
1956     els.saveState.textContent = "Cambios locales";
1957   }
1958 }
```

Código 9. Persistencia del estado y proyectos.

Clave	Contenido
levantamientos-topograficos-v1	Estado activo serializado.
levantamientos-topograficos-projects-v1	Mapa de proyectos guardados por nombre.
levantamientos-topograficos-current-project-v1	Nombre del proyecto actualmente abierto.

saveLocalState() se ejecuta durante update(). Si el nombre coincide con el proyecto cargado, también actualiza su copia nombrada. pushHistory(), undo() y redo() conservan instantáneas JSON; HISTORY_LIMIT limita la pila a 60 estados.

Riesgo operativo
localStorage depende del origen, perfil y dispositivo. No existe respaldo de servidor. La exportación CSV es el mecanismo de respaldo portátil.

12. Importación y exportación

Código 11. Exportación de coordenadas TXT - app.js

```
2212 function exportCoordinatesTxt() {
2213   const survey = computeSurvey();
2214   const points = survey.points.filter((point) => point.hasCoordinates);
2215   const protectValue = (value) => {
2216     const text = String(value ?? "");
2217     return /[",\r\n]/.test(text) ? `${text.replaceAll('"', '""')}` : text;
2218   };
2219   const rows = points.map((point, index) => [
2220     index + 1,
2221     formatNumber(point.east),
2222     formatNumber(point.north),
2223     0,
2224     state.zones.find((zone) => zone.id === point.zoneId)?.name || "",
2225   ]);
2226   const content = rows.map((row) => row.map(protectValue).join(", ").join("\r\n"));
2227   downloadFile(`\u0000${content}`, `TOPORAY_${safeFileName(state.projectName)}-coordenadas.txt`, "text/plain;charset=utf-8");
2228 }
```

Código 11. TXT con numeración global, coordenadas y zona.

Formato	Implementación
CSV	Incluye zona, color, tipo, observación, cálculo y estado. csvEscape() protege comillas, comas y saltos.
Importar CSV	Normaliza encabezados, admite columnas alternativas y reconstruye zonas y puntos.
TXT coordenadas	Filtra puntos válidos y renumera globalmente desde 1; Z es 0.
TXT cálculos	Describe conversión GMS, azimut, rumbo, proyecciones y coordenadas.
PNG	Convierte el canvas técnico a Blob de imagen/png.
Informe	Genera HTML de impresión con plantilla, plano y tablas por zona.

Codificación
CSV y TXT se descargan con marca BOM UTF-8 para mejorar la apertura en editores y hojas de cálculo de Windows.

13. Informe técnico y escala de dibujo

`drawingScaleDetails()` compara la extensión real del levantamiento con el área útil del papel. La escala automática selecciona el primer denominador normalizado que permite encajar el dibujo.

Parámetro	Valor o regla
Formatos	A0 841×1189, A1 594×841, A2 420×594, A3 297×420 y A4 210×297 mm.
Margen	10 mm por lado.
Cajetín	35 mm reservados en la dimensión inferior.
Orientación	Vertical u horizontal; intercambia ancho y alto.
Escalas	1:50 a 1:10000 en denominadores predefinidos.
Plano exportable	Fondo blanco, ejes balanceados, leyenda y BM.
Informe	Carta horizontal sobre plantilla TOPORAY, con gráfica y tablas ajustadas.

- La vista interactiva puede ser oscura o clara; la salida técnica siempre es blanca.
- El informe usa `escapeHtml()` para insertar nombres y descripciones.
- Las tablas de zona se distribuyen sin exceder el ancho del papel.
- `window.print()` permite seleccionar Guardar como PDF.
- El navegador debe permitir la ventana emergente del informe.

Verificación requerida

Después de cambiar la plantilla o CSS de impresión, genere un PDF real y renderícelo a imagen para comprobar recortes, fondos y legibilidad.

14. Eventos, accesibilidad y comportamiento móvil

Código 12. Cambio de vista y menú adaptable - app.js

```
2501 function switchView(view) {
2502   if (!els.viewPanels.some((panel) => panel.dataset.viewPanel === view)) return;
2503   activeView = view;
2504   els.viewPanels.forEach((panel) => panel.classList.toggle("is-active", panel.dataset.viewPanel === view));
2505   els.navButtons.forEach((button) => button.classList.toggle("is-active", button.dataset.view === view));
2506   els.mobileNavButtons.forEach((button) => button.classList.toggle("is-active", button.dataset.view === view));
2507   els.sidebar.classList.remove("is-mobile-open");
2508   els.sidebarBackdrop.classList.remove("is-visible");
2509   if (view === "plan" && state) window.requestAnimationFrame(() => drawPlot(computeSurvey()));
2510 }
2511
2512 function toggleSidebar() {
2513   if (window.matchMedia("(max-width: 980px)").matches) {
2514     const open = els.sidebar.classList.toggle("is-mobile-open");
2515     els.sidebarBackdrop.classList.toggle("is-visible", open);
2516     return;
2517   }
2518   const collapsed = els.sidebar.classList.toggle("is-collapsed");
2519   document.body.classList.toggle("sidebar-collapsed", collapsed);

```

Código 12. Cambio de vista y comportamiento del menú según el ancho.

Área	Práctica implementada
Eventos	Listeners centralizados al final de app.js; cada función modifica state y llama update().
Teclado	Botones, selects, inputs y dialog nativos conservan interacción estándar.
Etiquetas	aria-label en navegación, canvas, botones de icono y controles sin texto visible.
Mensajes	Toast con role=status y regiones aria-live para resultados dinámicos.
Responsive	Menú lateral móvil, navegación inferior y reorganización de formularios.
Tablas	Scroll horizontal con columnas de identificación y acciones fijadas.

Cuando se añadan controles nuevos, deben conservar un nombre accesible, estados disabled coherentes y áreas táctiles suficientes. No debe dependerse exclusivamente del color para comunicar errores.

15. Pruebas y control de calidad

Antes de publicar una versión se recomienda ejecutar la siguiente matriz mínima.

Grupo	Casos obligatorios
Validación	G=0/360; M y S en 0, 59.999 y 60; distancia negativa, 0 y positiva; número repetido.
Cálculo	Azimuts en los cuatro cuadrantes; proyecciones con signos esperados; base BM y referencia de punto.
Geometría	Polígono de 3 y 4 puntos; línea; punto aislado; cruce; duplicado; vacío intermedio.
Puntos	Editar, limpiar, reingresar, eliminar, reordenar y deshacer.
Persistencia	Guardar, recargar, abrir, duplicar, renombrar y eliminar proyecto.
Intercambio	CSV con comas/comillas; importación; TXT secuencial; Z=0.
Visual	Canvas cuadrado, BM fijo, zoom, arrastre, escritorio y 320/430/980/1600 px.
Salida	PNG blanco 1400×1000; informe PDF sin recortes ni fondos negros.

Comprobaciones automáticas recomendadas

- Ejecutar `node --check app.js`.
- Usar Playwright para navegar todas las vistas y registrar `pageerror`.
- Interceptar descargas y validar nombres, columnas y codificación.
- Leer píxeles de la gráfica exportada para comprobar fondo blanco.
- Renderizar el PDF de informe y revisar visualmente cada página.

16. Mantenimiento, seguridad y limitaciones

Mantenimiento

- Conserve STORAGE_KEY y las claves de proyectos o implemente una migración explícita.
- Actualice migrateState() cuando agregue propiedades persistentes.
- Mantenga computeSurvey() libre de efectos secundarios para facilitar pruebas.
- Añada formatos de exportación como funciones separadas que consuman computeSurvey().
- Pruebe la plantilla de informe cada vez que cambien columnas o tipografías.
- Actualice la versión visible, README y manuales en una misma entrega.

Seguridad y privacidad

- La aplicación no transmite datos por sí sola y no contiene autenticación.
- escapeHtml() debe usarse para cualquier texto insertado en el informe HTML.
- Los archivos importados deben tratarse como datos no confiables y validarse.
- Un despliegue institucional debe usar HTTPS y políticas de contenido apropiadas.

Limitaciones conocidas

Limitación	Implicación
Modelo 2D	Z se exporta como 0; no hay cálculo altimétrico.
Plano local	No hay transformación de datum, proyección cartográfica ni geodesia.
Persistencia local	No hay colaboración, cuentas, sincronización ni respaldo remoto.
Precisión	Los cálculos usan Number de JavaScript; el redondeo visible no cambia el valor interno.
Informe por navegador	El resultado final depende parcialmente del motor de impresión.

Criterio de liberación

Una versión debe publicarse solo después de validar cálculo, persistencia, exportaciones, comportamiento responsive y representación PDF con datos de prueba conocidos.